Lin_Timo_CSE 565_StructuralBasedTesting

Part 1

Tool & Coverage Types

Tool

JaCoCo (Java Code Coverage Library), run via the `-javaagent` at test time and `org.jacoco.cli` to generate an HTML report.

Coverage Types

- Statement Coverage
- Decision Coverage
- Line Coverage
- Method Coverage
- Class Coverage

Test Cases

Test name	Input	Expected result	Purpose / Branches hit
`candy_exact_cost`	(20, "candy")	`Item dispensed.`	`item=="candy"` true; `input==cost` branch
`coke_exact_cost`	(25, "coke")	`Item dispensed.`	`item=="coke"` true; `input==cost`
`coffee_exact_cost`	(45, "coffee")	`Item dispensed.`	`item=="coffee"` true; `input==cost`
`coke_more_than_cost_change_5`	(30, "coke")	change 5	`input>cost` branch
`coffee_more_than_cost_change_5`	(50, "coffee")	change 5	`input>cost` branch (different values)
`coffee_less_than_cost_purchase_candy_or_coke`	(44, "coffee")	"...missing 1... Can purchase candy or coke."	`input<cost` → nested `input<45` true

Test name	Input	Expected result	Purpose / Branches hit
`coke_less_than_cost_purchase_candy`	(24, "coke")	"...missing 1... Can purchase candy."	`input<25` true; `input<20` false
`coke_less_than_cost_cannot_purchase`	(19, "coke")	"...missing 6... Cannot purchase item."	`input<20` true (most restrictive)
`candy_less_than_cost_cannot_purchase`	(10, "candy")	"...missing 10... Cannot purchase item."	`input<20` true with different item
`unknown_item_equal_cost_zero`	(0, "tea")	`Item dispensed.`	All item checks false; `cost` stays 0; equals branch
`unknown_item_more_than_cost_zero`	(3, "tea")	change 3	Unknown item + `input>cost`
`touch_default_constructor`	new `VendingMachine()`	(no output)	Executes implicit constructor → avoids "Missed Methods: 1" (ensures 100% statements)

 alt text  alt text

Coverage Report & Discussion

- Statement Coverage: 100%
  - If the coverage is initially < 100%, add the `touch_default_constructor` test. JaCoCo counts the implicit default constructor unless executed.
- Decision (Branch) Coverage: 93%
  - In my code, the nested condition `if (input < 45)` sits inside the `else` where `input < cost`. Since `cost ≤ 45` (`candy=20`, `coke=25`, `coffee=45`), the case `input ≥ 45` and `input < cost` cannot occur (it would require `input ≥ 45` and `cost > input`, but max cost is 45). Therefore, the "false" edge of `input < 45` is logically infeasible.

 alt text

Part 2

Tool used & Features

I choose **PMD** as the code analyzer for this part. **PMD** reads and analyzes source files to detect potential coding problems and data-flow anomalies without executing the program. PMD uses a rule-based analysis engine that

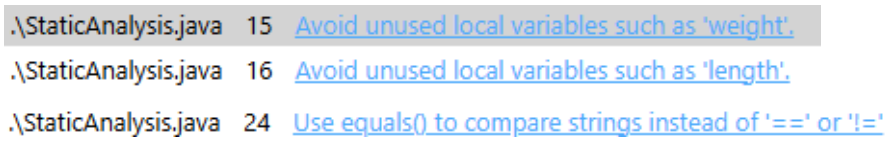
parses source files into an Abstract Syntax Tree (AST) and applies rules that look for patterns of bad practice, including:

- Unused local variables or parameters,
- Incorrect string comparison using `==` instead of `.equals()`,
- Unreachable or redundant code,
- Inefficient expressions, and
- Violations of common Java style and design principles.

For this project, the Quickstart ruleset (`rulesets/java/quickstart.xml`) was used. It includes general-purpose rules for detecting unused variables and incorrect equality comparisons—precisely matching the two intentional anomalies in the given program.

Description & Analysis of the Findings

Finding	Description	PMD Message / Rule	Type of Anomaly
1. **Unused local variables** in the constructor	In the `StaticAnalysis()` constructor, the local variables `weight` and `length` are declared and initialized but never used.	"Avoid unused local variables such as 'weight' and 'length'." *(Rule: UnusedLocalVariable)*	**Defined but never used** — a classic *dead store anomaly*.
2. **Incorrect string comparison using** `==`	In `calculateCost(int, int, String)`, the line `if (product == "Electronics")` uses the equality operator (`==`) to compare strings. In Java, `==` checks for reference equality rather than content equality.	"Use equals() to compare strings instead of '==' or '!='" *(Rule: UseEqualsToCompareStrings)*	**Wrong comparison operator** — a *use of inconsistent definition* anomaly.



Assessment of the Tool

Aspect	Assessment
Features and Functionalities	PMD provides a rich set of predefined rules that analyze variable definitions, control flow, code structure, and naming conventions. It supports customizable rule sets and multiple report formats (text, XML, HTML). The ability to detect data-flow anomalies, style issues, and complexity metrics makes it versatile for both academic and industrial use.
Type of Anomalies Covered	PMD detects **data-flow anomalies** (unused variables, redundant assignments, redefinitions), **syntactic anomalies** (incorrect operators, missing braces), and **structural issues** (unreachable code, complexity). It focuses on identifying potential defects that could cause incorrect program behavior or maintainability problems.

Aspect	Assessment
Ease of Use	PMD is lightweight, runs from a single command line, and does not require any build system (like Maven or Gradle). It can analyze both individual Java files and large projects. Reports are generated instantly and can be viewed in either terminal output or user-friendly HTML format, making the tool easy to use even for beginners.